1. (a) Extend the logical relation to support products

$$\mathsf{Halt}_{X\times Y} = \{e | \mathsf{fst} e \in \mathsf{Halt}_X \wedge \mathsf{snd} e \in \mathsf{Halt}_Y\}$$

(b) Extend the logical relation to support sums

$$\mathsf{Halt}_{X+Y} = \{e | \exists v. e \to v \wedge \forall e_1, e_2 \in \mathsf{Halt}_C. \mathsf{case}(e, Lx \to e_2,) \in \mathsf{Halt}_C\}$$

2. (a) Prove the regularity lemma

RTP: if $\Theta \vdash \Gamma\ \mathsf{ctx}$ and $\Theta; \Gamma \vdash e : A$, then $\Theta \vdash A\ \mathsf{type}$.
That is, $\forall \Theta, \Gamma, e, A.\ \Theta \vdash \Gamma\ \mathsf{ctx} \wedge \Theta; \Gamma \vdash e : A \Rightarrow \Theta \vdash A\ \mathsf{type}$.
We proceed by rule induction on $\Theta; \Gamma \vdash e : A$, defining

$$\Phi(\Theta, \Gamma, e, A) \triangleq \Theta \vdash \Gamma\ \mathsf{ctx} \Rightarrow \Theta \vdash A\ \mathsf{type}$$

We consider the various cases.
Variable case:

$$\frac{x : A \in \Gamma}{\Theta; \Gamma \vdash x : A}$$

Suppose $\Theta \vdash \Gamma\ \mathsf{ctx}$ (1)
and $x : A \in \Gamma$. (2)
RTP: $\Theta \vdash A\ \mathsf{type}$.
By (1) and (2), $\Theta \vdash \Gamma', x : A\ \mathsf{ctx}$, where $\Gamma = \Gamma', x : A, \Gamma''$.
By inversion, $\Theta \vdash \Gamma'\ \mathsf{ctx} \wedge \Theta \vdash A\ \mathsf{type}$.
Abstraction case:

$$\frac{\Theta \vdash A\ \mathsf{type} \quad \Theta; \Gamma, x : A \vdash e : B}{\Theta; \Gamma \vdash \lambda x : A.\, e : A \to B}$$

Suppose that $\Phi(\Theta, (\Gamma, x : A), e, B)$ holds, that is, $\Theta \vdash \Gamma, x : A\ \mathsf{ctx} \Rightarrow \Theta \vdash B\ \mathsf{type}$. (3)
Further suppose $\Theta \vdash A\ \mathsf{type}$. (1)
RTP: $\Phi(\Theta, \Gamma, \lambda x : A.\, e, A \to B)$ holds.
Suppose $\Theta \vdash \Gamma\ \mathsf{ctx}$. (2)
RTP: $\Theta \vdash A \to B\ \mathsf{type}$.
By (1) and (2), $\Theta \vdash \Gamma, x : A\ \mathsf{ctx}$. (4)
By (3) and (4), $\Theta \vdash B\ \mathsf{type}$. (5)
By (5) and (1), $\Theta \vdash A \to B\ \mathsf{type}$.
Application case:

$$\frac{\Theta; \Gamma \vdash e : A \to B \quad \Theta; \Gamma \vdash e' : A}{\Theta; \Gamma \vdash e\ e' : B}$$

Suppose $\Phi(\Theta, \Gamma, e, A \to B) \wedge \Phi(\Theta, \Gamma, e', A)$.
RTP: $\Phi(\Theta, \Gamma, e\ e', B)$, that is, $\Theta \vdash \Gamma\ \mathsf{ctx} \Rightarrow \Theta \vdash B\ \mathsf{type}$.
Suppose $\Theta \vdash \Gamma\ \mathsf{ctx}$. Then, $\Theta \vdash A \to B\ \mathsf{type} \wedge \Theta \vdash A\ \mathsf{type}$. (1)
By (1), $\Theta \vdash A\ \mathsf{type} \wedge \Theta \vdash B\ \mathsf{type}$.
Type abstraction case:

$$\frac{\Theta, \alpha; \Gamma \vdash e : B}{\Theta; \Gamma \vdash \Lambda\alpha.\, e : \forall \alpha.\, B}$$

IH: $\Phi((\Theta, \alpha), \Gamma, e, B)$, that is, $\Theta, \alpha \vdash \Gamma\ \mathsf{ctx} \Rightarrow \Theta, \alpha \vdash B\ \mathsf{type}$. (2)
RTP: $\Phi(\Theta, \Gamma, \Lambda\alpha.\, e, \forall \alpha.\, B)$, that is, $\Theta \vdash \Gamma\ \mathsf{ctx} \Rightarrow \Theta \vdash \forall \alpha.\, B\ \mathsf{type}$.
Suppose $\Theta \vdash \Gamma\ \mathsf{ctx}$. (1)

By (1) and weakening, $\Theta, \alpha \vdash \Gamma$ ctx. (3)
By (2) and (3), $\Theta, \alpha \vdash B$ type. (4)
By (4), $\Theta \vdash \forall \alpha.\, B$ type.
Type application case:

$$\frac{\Theta; \Gamma \vdash e : \forall \alpha.\, B \quad \Theta \vdash A \text{ type}}{\Theta; \Gamma \vdash e\ A : [A/\alpha]B}$$

IH: $\Phi(\Theta, \Gamma, e, \forall \alpha.\, B)$, that is, $\Theta \vdash \Gamma$ ctx $\Rightarrow \Theta \vdash \forall \alpha.\, B$ type. (1)
Assume $\Theta \vdash A$ type. (2)
RTP: $\Phi(\Theta, \Gamma, e\ A, [A/\alpha]B)$.
Assume $\Theta \vdash \Gamma$ ctx. (3)
RTP: $\Theta \vdash [A/\alpha]B$ type.
By (3) and (1), $\Theta \vdash \forall \alpha.\, B$ type. (4)
By (4) and inversion, $\Theta, \alpha \vdash B$ type. (5)
By (5), (2) and substitution, $\Theta \vdash [A/\alpha]B$ type.

(b) Define a Church Encoding for the unit type

$\forall \alpha. \alpha \to \alpha$

(c) Define a Church Encoding for the empty type

$\forall \alpha. \alpha.$

(d) Define a Church Encoding for the binary trees

$\forall \alpha. \alpha \to (X \to \alpha \to \alpha \to \alpha) \to \alpha$

3. (a) Prove the other direction of the closure property of the $\Theta \vdash \forall \alpha. A\text{type}$ case.

RTP: if $e \to e'$ and $e \in [\![\Theta \vdash \forall \alpha. A\text{type}]\!]\theta$, then $e' \in [\![\Theta \vdash \forall \alpha. A\text{type}]\!]\theta$
We note that we are using rule induction on the well-formed property of a type in order to prove this property. Hence, we can define

$$\Phi(\Theta, A) \triangleq e \to e' \Rightarrow e \in [\![\Theta \vdash \forall \alpha. A\text{type}]\!]\theta \Leftrightarrow e' \in [\![\Theta \vdash \forall \alpha. A\text{type}]\!]\theta$$

Assume $\Phi(\Theta, \alpha, A)$. RTP: $\Phi(\Theta, \forall \alpha. A)$
Suppose $e \to e'$ and $e \in [\![\Theta \vdash \forall \alpha. A\text{type}]\!]\theta$
By definition, $\forall C, X, e\ C \in [\![\Theta, \alpha \vdash A\text{type}]\!](\theta, X/\alpha)$
Let C, X be arbitrary. By the congruence rule, $e\ C \to e'C$. By the induction hypothesis then, $e\ C \in [\![\Theta, \alpha \vdash A\text{type}]\!](\theta, X/\alpha)$. Hence, by definition, $e' \in [\![\Theta \vdash \forall \alpha. A\text{type}]\!]\theta$

(b) Prove the other direction of the substitution property of the $\Theta \vdash \forall \alpha. A\text{type}$ case.

RTP: If $\Theta \vdash A$ type and $\Theta, \alpha \vdash B$ type, then $[\![\Theta \vdash [A/\alpha]B \text{ type}]\!]\theta = [\![\Theta, \alpha \vdash B \text{ type}]\!](\theta, [\![\Theta \vdash A \text{ type}]\!]\theta/\alpha)$
We proceed by induction on the well-formed property of a type. We define:

$$\begin{aligned}\Phi(\Theta, B) \triangleq\ & \Theta \vdash A \text{ type} \wedge \Theta, \alpha \vdash B \text{ type} \Rightarrow \\ & [\![\Theta \vdash [A/\alpha]B \text{ type}]\!]\theta \\ & = [\![\Theta, \alpha \vdash B \text{ type}]\!](\theta, [\![\Theta \vdash A \text{ type}]\!]\theta/\alpha)\end{aligned}$$

We assume $\Phi((\Theta, \beta), B)$. We want to prove $\Phi(\Theta, \forall\beta.\, B)$.
(⇒) Suppose

$$e \in [\![\Theta \vdash [A/\alpha](\forall\beta.\, B)\ \mathsf{type}]\!]\theta \tag{1}$$

RTP: $e \in [\![\Theta, \alpha \vdash \forall\beta.\, B\ \mathsf{type}]\!](\theta, [\![\Theta \vdash A\ \mathsf{type}]\!]\theta/\alpha)$, that is, by definition, for all C, X,

$$e\ C \in [\![\Theta, \alpha, \beta \vdash B\ \mathsf{type}]\!] \\ (\theta, [\![\Theta \vdash A\ \mathsf{type}]\!]\theta/\alpha, X/\beta)$$

By (1) and definition, for all D, Y,

$$e\ D \in [\![\Theta, \beta \vdash [A/\alpha]B\ \mathsf{type}]\!](\theta, Y/\beta)$$

Let D, Y be arbitrary. Then

$$e\ D \in [\![\Theta, \beta \vdash [A/\alpha]B\ \mathsf{type}]\!](\theta, Y/\beta) \tag{2}$$

By (2) and induction,

$$e\ D \in [\![\Theta, \beta, \alpha \vdash B\ \mathsf{type}]\!] \\ (\theta, Y/\beta, [\![\Theta, \beta \vdash A\ \mathsf{type}]\!](\theta, Y/\beta)/\alpha)$$

By weakening and exchange,

$$e\ D \in [\![\Theta, \alpha, \beta \vdash B\ \mathsf{type}]\!] \\ (\theta, [\![\Theta \vdash A\ \mathsf{type}]\!]\theta/\alpha, Y/\beta)$$

Since D, Y were arbitrary, this is exactly the RTP.

(c) Prove the fundamental lemma for the for-all introduction case.

We proceed by rule induction on the typing relation. We define $\Phi(\vec{\alpha}, \overrightarrow{x_i : A_i}, e, B)$ to be such that for all $\theta, \vec{e}, \vec{C}$, if θ interprets $\vec{\alpha}$, $\vec{\alpha} \vdash \overrightarrow{x_i : A_i}\ \mathsf{ctx}$ and $\forall x_i : A_i \in \overrightarrow{x_i : A_i}.\ e_i \in [\![\Theta \vdash A_i\ \mathsf{type}]\!]\theta$, then

$$[C_1/\alpha_1, \ldots, C_k/\alpha_k][e_1/x_1, \ldots, e_n/x_n]\, e \in [\![\Theta \vdash B\ \mathsf{type}]\!]\theta$$

We consider the rule for type abstraction:

$$\frac{\Theta, \alpha; \Gamma \vdash e : A}{\Theta; \Gamma \vdash \Lambda\alpha.\, e : \forall\alpha.\, A}$$

Assume $\Phi((\Theta, \alpha), \Gamma, e, A)$, that is, for all $\theta, \vec{e}, \vec{C}$, if θ interprets (Θ, α), $\Theta, \alpha \vdash \Gamma\ \mathsf{ctx}$ and $\forall x_i : A_i \in \Gamma.\ e_i \in [\![\Theta, \alpha \vdash A_i\ \mathsf{type}]\!]\theta$, then

$$[C_1/\alpha_1, \ldots, C_k/\alpha_k, C_{k+1}/\alpha][e_1/x_1, \ldots, e_n/x_n]\, e \in [\![\Theta, \alpha \vdash A\ \mathsf{type}]\!]\theta$$

RTP: $\Phi(\Theta, \Gamma, \Lambda\alpha.\, e, \forall\alpha.\, A)$.
Let $\theta', \vec{e}, \vec{C}$ be arbitrary. Suppose $\Theta \vdash \Gamma\ \mathsf{ctx}$, (1)
θ' interprets Θ, (2)
and $\forall x_i : A_i \in \Gamma.\ e_i \in [\![\Theta \vdash A_i\ \mathsf{type}]\!]\theta'$.
RTP:

$$[C_1/\alpha_1, \ldots, C_k/\alpha_k][e_1/x_1, \ldots, e_n/x_n]\, \Lambda\alpha.\, e \in [\![\Theta \vdash \forall\alpha.\, A\ \mathsf{type}]\!]\theta'$$

Equivalently, prove that for all types B and semantic types X,

$$[C_1/\alpha_1, \ldots, C_k/\alpha_k, B/\alpha][e_1/x_1, \ldots, e_n/x_n]\, e \in [\![\Theta, \alpha \vdash A\ \mathsf{type}]\!](\theta', X/\alpha)$$

By (1) and weakening, $\Theta, \alpha \vdash \Gamma\ \mathsf{ctx}$.
Let X, B be arbitrary. $(\theta', X/\alpha)$ interprets (Θ, α) since (2).
By the induction hypothesis,

$$[C_1/\alpha_1, \ldots, C_k/\alpha_k, B/\alpha][e_1/x_1, \ldots, e_n/x_n]\, e \in [\![\Theta, \alpha \vdash A\ \mathsf{type}]\!](\theta', X/\alpha)$$

and we are done.

1 2024 Paper 8 Question 13

https://www.cl.cam.ac.uk/teaching/exams/pastpapers/y2024p8q13.pdf

Consider the simply-typed lambda calculus with only function types and boolean types, with true, false, and if-then-else term formers for the boolean type.

(a) Define a logical relation suitable for establishing the termination of programs in this language. [4 marks]

We note that the types in this language is defined such that:

$$T := \mathbb{B} \mid A \to B$$

We hence denote the logical relation as such:

$$\mathsf{Halt}_{\mathbb{B}} = \{e | \exists v \in \mathbb{B}. e \to^* v\}$$
$$\mathsf{Halt}_{A \to B} = \{e | \exists v \in A \to B. e \to^* v \wedge \forall e' \in \mathsf{Halt}_A, ee' \in \mathsf{Halt}_B\}$$

(b) State the closure property of the logical relation. [2 marks]

If $e \to e'$, then $e \in \mathsf{Halt}_A$ if and only if $e' \in \mathsf{Halt}_A$.

(c) Prove closure for the case of the boolean type. [6 marks]

RTP: if $e \to e'$, then $e \in \mathsf{Halt}_{\mathbb{B}}$ if and only if $e' \in \mathsf{Halt}_{\mathbb{B}}$.
Suppose $e \to e'$. We show $e \in \mathsf{Halt}_{\mathbb{B}} \Leftrightarrow e' \in \mathsf{Halt}_{\mathbb{B}}$.
($\Rightarrow$) Suppose that $e \in \mathsf{Halt}_{\mathbb{B}}$. We want to show that $e' \in \mathsf{Halt}_{\mathbb{B}}$. By definition, $e \to^* \mathsf{True}$ or $e \to^* \mathsf{False}$. We approach this by cases.
Case 1: $e \to^* \mathsf{True}$. By the assumption that $e \to e'$, e is not a value, so $e \to e''$ and $e'' \to^* \mathsf{True}$ for some e''. As the simply typed lambda calculus is deterministic, we can conclude that $e'' = e'$, so $e' \to^* \mathsf{True}$.
Case 2: $e \to^* \mathsf{False}$. Analogous to Case 1.
($\Leftarrow$) Suppose that $e' \in \mathsf{Halt}_{\mathbb{B}}$. We want to show that $e \in \mathsf{Halt}_{\mathbb{B}}$. By definition, $e' \to^* \mathsf{True}$ or $e' \to^* \mathsf{False}$. By our assumption that $e \to e'$, it follows that $e \to^* \mathsf{True}$ or $e \to^* \mathsf{False}$.

(d) State the fundamental lemma for this language. [2 marks]

The fundamental lemma for this language would state:
If $x_1 : A_1, \ldots, x_n : A_n \vdash e : C \wedge \forall i \in \{1, \ldots, n\}. v_i \in \mathsf{Halt}_{A_i}$, then $[v_1/x_1, \ldots, v_n/x_n]e \in \mathsf{Halt}_C$

(e) Prove the fundamental lemma for the if-then-else case. [6 marks]

To prove the fundamental lemma, we proceed by rule induction on the inductively defined set for the simply typed lambda calculus with only function types and boolean types. We define our induction hypothesis to be:

$$\Phi(\Gamma, e, A) \triangleq \Gamma = x_1 : A_1, \ldots, x_n : A_n \wedge (\forall i \in \{1, \ldots, n\}.\ v_i \in \mathsf{Halt}_{A_i}) \Rightarrow [v_1/x_1, \ldots, v_n/x_n]\, e \in \mathsf{Halt}_A$$

We consider the rule for if-then-else:

$$\frac{\Gamma \vdash e_1 : \mathbb{B} \quad \Gamma \vdash e_2 : X \quad \Gamma \vdash e_3 : X}{\Gamma \vdash \mathsf{if}\ e_1\ \mathsf{then}\ e_2\ \mathsf{else}\ e_3 : X}$$

We assume $\Phi(\Gamma, e_1, \mathbb{B})$, $\Phi(\Gamma, e_2, X)$ and $\Phi(\Gamma, e_3, X)$.
RTP: $\Phi(\Gamma, \mathsf{if}\ e_1\ \mathsf{then}\ e_2\ \mathsf{else}\ e_3, X)$.
Suppose $\Gamma = x_1 : A_1, \ldots, x_n : A_n$ and $\forall i \in \{1, \ldots, n\}.\ v_i \in \mathsf{Halt}_{A_i}$. By the induction hypotheses,

$$[v_1/x_1, \ldots, v_n/x_n]\, e_1 \in \mathsf{Halt}_{\mathbb{B}}$$
$$[v_1/x_1, \ldots, v_n/x_n]\, e_2 \in \mathsf{Halt}_X$$
$$[v_1/x_1, \ldots, v_n/x_n]\, e_3 \in \mathsf{Halt}_X$$

By definition, $[v_1/x_1, \ldots, v_n/x_n]\, e_1 \to^* \mathsf{True}$ or $[v_1/x_1, \ldots, v_n/x_n]\, e_1 \to^* \mathsf{False}$. We proceed by cases.
Case 1: $[v_1/x_1, \ldots, v_n/x_n]\, e_1 \to^* \mathsf{True}$. Then

$$\begin{aligned} &[v_1/x_1, \ldots, v_n/x_n]\, (\mathsf{if}\ e_1\ \mathsf{then}\ e_2\ \mathsf{else}\ e_3) \\ &\to^* [v_1/x_1, \ldots, v_n/x_n]\, (\mathsf{if}\ \mathsf{True}\ \mathsf{then}\ e_2\ \mathsf{else}\ e_3) && \text{by the congruence rule} \\ &\to [v_1/x_1, \ldots, v_n/x_n]\, e_2 && \text{by the reduction rule} \end{aligned}$$

By the closure rule, since $[v_1/x_1, \ldots, v_n/x_n]\, e_2 \in \mathsf{Halt}_X$, we have $[v_1/x_1, \ldots, v_n/x_n]\, (\mathsf{if}\ e_1\ \mathsf{then}\ e_2\ \mathsf{else}\ e_3) \in \mathsf{Halt}_X$.
Case 2: $[v_1/x_1, \ldots, v_n/x_n]\, e_1 \to^* \mathsf{False}$. Analogous to Case 1.

2 2023 Paper 8 Question 13

https://www.cl.cam.ac.uk/teaching/exams/pastpapers/y2023p8q13.pdf

Consider Gödel's T, the simply-typed lambda calculus with function and natural number types, with zero, successor and iterator term formers for the natural number type.

(a) Define a logical relation suitable for establishing the termination of closed programs in this language. [5 marks]

We note that the types in this language is defined such that:

$$T := \mathbb{N} \mid A \to B$$

We hence denote the logical relation as such:

$$\mathsf{R}_{\mathbb{N}} = \{e | \exists k \in \mathbb{N}. e \to^* s^k(z)\}$$
$$\mathsf{R}_{A \to B} = \{e | \exists v \in A \to B. e \to^* v \wedge \forall e' \in \mathsf{R}_A, ee' \in \mathsf{R}_B\}$$

(b) State the fundamental lemma for this language. [3 marks]

The fundamental lemma for this language would state:
If $x_1 : A_1, \ldots, x_n : A_n \vdash e : C \wedge \forall i \in \{1, \ldots, n\}. v_i \in \mathsf{R}_{A_i}$, then $[v_1/x_1, \ldots, v_n/x_n]e \in \mathsf{R}_C$

(c) State formally what it means for a set of terms X to be "closed under reduction". [2 marks]

If $e \to e'$, then $e \in X$ if and only if $e' \in X$.

(d) Prove the fundamental lemma holds for the iterator case. [10 marks]

To prove the fundamental lemma, we proceed by rule induction on the inductively defined set for the simply typed lambda calculus with only function types and natural number types. We define our induction hypothesis to be:

$$\Phi(\Gamma, e, A) \triangleq \Gamma = x_1 : A_1, \ldots, x_n : A_n \wedge (\forall i \in \{1, \ldots, n\}.\ v_i \in \mathsf{R}_{A_i}) \Rightarrow [v_1/x_1, \ldots, v_n/x_n]\, e \in \mathsf{R}_A$$

We consider the rule for the iterator:

$$\frac{\Gamma \vdash e_1 : \mathbb{N} \quad \Gamma \vdash e_2 : X \quad \Gamma, x : X \vdash e_3 : X}{\Gamma \vdash \mathsf{iter}(e_1,\ z \to e_2,\ s(x) \to e_3) : X}$$

We assume $\Phi(\Gamma, e_1, \mathbb{N})$, $\Phi(\Gamma, e_2, X)$ and $\Phi((\Gamma, x : X), e_3, X)$.
RTP: $\Phi(\Gamma, \mathsf{iter}(e_1,\ z \to e_2,\ s(x) \to e_3), X)$.
Suppose $\Gamma = x_1 : A_1, \ldots, x_n : A_n$ and $\forall i \in \{1, \ldots, n\}.\ v_i \in \mathsf{R}_{A_i}$. By the induction hypotheses,

$$[v_1/x_1, \ldots, v_n/x_n]\, e_1 \in \mathsf{R}_{\mathbb{N}}$$
$$[v_1/x_1, \ldots, v_n/x_n]\, e_2 \in \mathsf{R}_X$$
$$[v_1/x_1, \ldots, v_n/x_n, v/x]\, e_3 \in \mathsf{R}_X \quad \text{for all } v \in \mathsf{R}_X$$

By definition of $\mathsf{R}_{\mathbb{N}}$, $[v_1/x_1, \ldots, v_n/x_n]\, e_1 \to^* s^k(z)$ for some k. Hence, by the congruence rules,

$$[v_1/x_1, \ldots, v_n/x_n]\, \mathsf{iter}(e_1,\ z \to e_2,\ s(x) \to e_3) \to^* [v_1/x_1, \ldots, v_n/x_n]\, \mathsf{iter}(s^k(z),\ z \to e_2,\ s(x) \to e_3)$$

By closure, we can equivalently prove the result by proving that $[v_1/x_1, \ldots, v_n/x_n]\, \mathsf{iter}(s^k(z),\ z \to e_2,\ s(x) \to e_3) \in \mathsf{R}_X$. To prove this, we proceed by mathematical induction on k.
Base case: $k = 0$. By the reduction rules,

$$[v_1/x_1, \ldots, v_n/x_n]\, \mathsf{iter}(z,\ z \to e_2,\ s(x) \to e_3) \to [v_1/x_1, \ldots, v_n/x_n]\, e_2$$

and $[v_1/x_1, \ldots, v_n/x_n]\, e_2 \in \mathsf{R}_X$ by the induction hypothesis for e_2, so by closure we are done.
Inductive case: $k = m + 1$. Assume that $[v_1/x_1, \ldots, v_n/x_n]\, \mathsf{iter}(s^m(z),\ z \to e_2,\ s(x) \to e_3) \in \mathsf{R}_X$. We want to prove that $[v_1/x_1, \ldots, v_n/x_n]\, \mathsf{iter}(s^{m+1}(z),\ z \to e_2,\ s(x) \to e_3) \in \mathsf{R}_X$. By the reduction rules,

$$[v_1/x_1, \ldots, v_n/x_n]\, \mathsf{iter}(s^{m+1}(z),\ z \to e_2,\ s(x) \to e_3) \to [v_1/x_1, \ldots, v_n/x_n]\, [\mathsf{iter}(s^m(z),\ z \to e_2,\ s(x) \to e_3)/x]\, e_3$$

Let $v = [v_1/x_1, \ldots, v_n/x_n]\,\mathsf{iter}(s^m(z),\ z \to e_2,\ s(x) \to e_3)$. By the inner induction hypothesis, $v \in \mathsf{R}_X$, so by the induction hypothesis for e_3, $[v_1/x_1, \ldots, v_n/x_n, v/x]\,e_3 \in \mathsf{R}_X$. This is exactly the term above, so by closure we are done.

3 2021 Paper 9 Question 15

https://www.cl.cam.ac.uk/teaching/exams/pastpapers/y2021p9q15.pdf

(a) In a simply-typed lambda calculus augmented with first-class continuations, booleans, a list type and its iterator (i.e., fold, but not full recursion), write a function

$$\mathsf{every} : (X \to \mathsf{Bool}) \to \mathsf{List}\,X \to \mathsf{Bool}$$

such that every p xs returns true if every element of xs satisfies p, and false otherwise. This function should also stop iterating over the list as soon as it finds a false element. You may use SML- or OCaml-style notation if desired, but explain any notation used beyond the basic lambda calculus. [4 marks]

let every p xs = fold xs ([] → True) (x::r → if p x then r else throw False)

(b) In the monadic lambda calculus with state, suppose we change the typing rule for reading locations to not cause a monadic effect: If we suggest changing the monadic lambda calculus to permit treating reads as pure:

$$\frac{l : X \in \Sigma}{\Sigma; \Gamma \vdash\ !l : X}$$

(i) Is this rule still typesafe? Informally but carefully justify your answer. [2 marks]

This question is asking given this rule, would the inductively defined set still have the type safety property. Specifically, would it still have progress and type preservation? Yes, I think the monoids and the split between pure and inpure code only deals with termination, not with type safety.

(ii) Is the following *common subexpression elimination* transformation sound? Either give an argument why it is, or supply a counterexample and explain why it shows it is not. [6 marks]

```
let x = return e1;          let x = return e1;
let y = e2;         =====>  let y = e2;
let z = return e1;          [z/x]e3
e3
```

No. This code is run sequentially, meaning the effects of `y` would have effects on the result of `z`, and hence `x` and `z` could return different values, and when used in `e3`, would result in different results. Consider

```
let x = return !l;
let y = l := 0;
let z = return !l;
e3
```

This would not return the same result if location `l` initially stored a value other than `0`.

(c) In System F augmented with existential types, give an existential type for the interface of the natural numbers, and give an implementation for it. [8 marks]

For the natural numbers, we need some implementation of the zero, successor and iteration. To do this, we might provide an interface like:

$$\exists\alpha.\alpha \times (\alpha \to \alpha) \times (\forall\beta.\alpha \to \beta \to (\beta \to \beta) \to \beta)$$

We might expect an implementation like the church numerals, where we let:
$\mathsf{type} \triangleq \forall x.x \to (x \to x) \to x$
$\mathsf{zero} \triangleq \Lambda\alpha.\lambda z : \alpha.\lambda s : (\alpha \to \alpha).z$
$\mathsf{succ} \triangleq \lambda x : \mathsf{Nat}.\Lambda\alpha.\lambda z : \alpha.\lambda s : (\alpha \to \alpha).s(x\mathsf{Nat}zs)$
$\mathsf{iter} \triangleq \Lambda\alpha.\lambda x : \mathsf{Nat}.\lambda z : \alpha.\lambda s : \alpha \to \alpha.x\alpha zs$

4 2019 Paper 9 Question 14

https://www.cl.cam.ac.uk/teaching/exams/pastpapers/y2019p9q14.pdf

(a) In System F, give a Church encoding for (i) the Boolean type, (ii) the definition of the True and False constants, and (iii) the type and definition of the if-then-else operation. [3 marks]

(i) $\forall\alpha.\alpha \to \alpha \to \alpha$

(ii) True: $\Lambda\alpha.\lambda x : \alpha.\lambda y : \alpha.x$
False: $\Lambda\alpha.\lambda x : \alpha.\lambda y : \alpha.y$

(iii) if e_1 then e_2 else e_3. This will be of type: $\forall\alpha : B \to \alpha \to \alpha \to \alpha$. We can further define this as:

$$\Lambda\alpha.\lambda b : B.\lambda t : \alpha.\lambda f : \alpha.b\ \alpha\ t\ f$$

(b) In System F, give (i) a Church encoding Nat for the natural numbers, (ii) a Church encoding for the Zero : Nat and Succ : Nat → Nat constructors, and (iii) a type and definition for the iteration operator Iter for natural numbers. [3 marks]

(i) $\forall\alpha.\alpha \to (\alpha \to \alpha) \to \alpha$

(ii) Zero: $\Lambda\alpha.\lambda z : \alpha.\lambda s : \alpha \to \alpha.z$
Succ: $\lambda x : Nat.\Lambda\alpha.\lambda z : \alpha.\lambda s : \alpha \to \alpha.s(x\alpha zs)$

(iii) Iter(e_1, Zero → e_2, Succ (x) → e_3). The type of iter is hence $\forall\alpha.Nat \to \alpha \to (\alpha \to \alpha) \to \alpha$. A definition for this operator is thus:

$$\Lambda\alpha.\lambda x : Nat.z : \alpha.s : \alpha \to \alpha.x\ \alpha\ z\ s$$

(c) (i) In System F, give a Church encoding for (i) an Option_A type, (ii) the definitions of the $\mathsf{None} : \mathsf{Option}_A$ and $\mathsf{Some} : A \to \mathsf{Option}_A$ operations, and (iii) the type and definition of the case operation on options. ‘’

i. $\forall\alpha.\alpha \to (A \to \alpha) \to \alpha$

ii. None: $\Lambda\alpha.\lambda n : \alpha.\lambda s : (A \to \alpha).n$
Some: $\lambda x : A.\Lambda\alpha.\lambda n : \alpha.\lambda s : (A \to \alpha).sx$

iii. Case(e_1, None → e_2, Some(x) → e_3). The type of the case operation can be given as: $\forall\alpha.\text{Option}_A \to \alpha \to (A \to \alpha) \to \alpha$. The definition of the case operation can be given as:

$$\Lambda\alpha.\, : \mathrm{Option}_A.\lambda n : \alpha.\lambda s : (A \to \alpha).o\ \alpha\ n\ s$$

(ii) Assume that $n : B$ and $s : A \to B$, and then

(A) Prove that $\mathsf{Case}\,[B]\ n\ s\ \mathsf{None} = n$

$$\begin{aligned}
&\mathsf{Case}\,[B]\ \mathsf{None}\ n\ s \\
&= (\Lambda\alpha.\,\lambda o{:}\mathsf{Option}_A.\,\lambda n{:}\alpha.\,\lambda s{:}(A \to \alpha).\ o\ \alpha\ n\ s)\,[B]\ \mathsf{None}\ n\ s \\
&= \mathsf{None}\,[B]\ n\ s \\
&= n
\end{aligned}$$

(B) Prove that $\mathsf{Case}\,[B]\ n\ s\ (\mathsf{Some}\ x) = s\ x$ [5 marks]

$$\begin{aligned}
&\mathsf{Case}\,[B]\ (\mathsf{Some}\ x)\ n\ s \\
&= (\Lambda\alpha.\,\lambda o{:}\mathsf{Option}_A.\,\lambda n{:}\alpha.\,\lambda s{:}(A \to \alpha).\ o\ \alpha\ n\ s)\,[B]\ (\mathsf{Some}\ x)\ n\ s \\
&= (\mathsf{Some}\ x)\,[B]\ n\ s \\
&= (\Lambda\alpha.\,\lambda n{:}\alpha.\,\lambda s{:}(A \to \alpha).\ s\ x)\,[B]\ n\ s \\
&= s\ x
\end{aligned}$$

(d) In System F, define a predecessor operation $\mathsf{Pred} : \mathsf{Nat} \to \mathsf{Nat}$, which returns Zero if given Zero as an argument, and return n if given $\mathsf{Succ}\ n$ as an argument. [*Hint*: The option type may be useful in formulating this definition.] [8 marks]

We can define Pred with the following helper function. We first define

$$\mathsf{Helper} = \lambda x{:}\mathsf{Nat}.\ \mathsf{Iter}\,[\mathsf{Option}_{\mathsf{Nat}}]\ x\ \mathsf{None}\ (\lambda y{:}\mathsf{Option}_{\mathsf{Nat}}.\ \mathsf{Case}\,[\mathsf{Option}_{\mathsf{Nat}}]\ y\ (\mathsf{Some}\ \mathsf{Zero})\ (\lambda n{:}\mathsf{Nat}.\ \mathsf{Some}\ (\mathsf{Succ}\ n)))$$

We can then define Pred:

$$\mathsf{Pred} = \lambda x{:}\mathsf{Nat}.\ \mathsf{Case}\,[\mathsf{Nat}]\ (\mathsf{Helper}\ x)\ \mathsf{Zero}\ (\lambda y{:}\mathsf{Nat}.\ y)$$

(e) In System F, define a subtraction operator $\mathsf{Sub} : \mathsf{Nat} \to \mathsf{Nat} \to \mathsf{Nat}$, which is defined to be *saturating*. That is, $\mathsf{Sub}\ m\ n$ returns the difference if $m \geq n$, and returns 0 otherwise. [1 mark]

We can define the subtraction operator as such:

$$\mathsf{Sub} = \lambda x{:}\mathsf{Nat}.\ \lambda y{:}\mathsf{Nat}.\ y\,[\mathsf{Nat}]\ x\ \mathsf{Pred}$$

5 2018 Paper 8 Question 14

(a) Give typing rules for the introduction form $\mathsf{pack}(\tau, M)$ and elimination form $\mathsf{unpack}\ M\ \mathsf{as}$ of the existential type $\exists\alpha(\tau)$. [4 marks]

https://www.cl.cam.ac.uk/teaching/exams/pastpapers/y2018p8q14.pdf

$$\frac{\Theta \vdash \tau' \text{ type} \quad \Theta,\alpha \vdash \tau \text{ type} \quad \Theta;\Gamma \vdash M:[\tau'/\alpha](\tau)}{\Theta;\Gamma \vdash \text{pack}(\tau',M):\exists\alpha(\tau)}\ \exists I$$

$$\frac{\Theta;\Gamma \vdash M:\exists\alpha(\tau) \quad \Theta,\alpha;\Gamma,x:\tau \vdash N:C \quad \Theta \vdash c \text{ type}}{\Theta;\Gamma \vdash \text{ unpack } M \text{as } (x,\alpha) \text{ in } N:C}\ \exists E$$

(b) An infinite stream of booleans can be represented in the polymorphic lambda calculus using the existential type

$$\mathsf{stream} \triangleq \exists\alpha(\alpha \times (\alpha \to (\mathsf{bool} \times \alpha)))$$

(i) Using the encoding above, define a function $\mathsf{head} : \mathsf{stream} \to \mathsf{bool}$. [3 marks]

head can be defined as

$$\lambda s : \text{ stream. unpack} s \text{ as } (x,\alpha) \text{ in } fst(snd(x)(fst(x)))$$

(ii) Using the encoding above, define a function $\mathsf{tail} : \mathsf{stream} \to \mathsf{stream}$. [3 marks]

tail can be defined as

$$\lambda s : \text{ stream. unpack} s \text{ as } (x,\alpha) \text{ in pack}(\alpha,(snd(snd(x)fst(x)),snd(x)))$$

(iii) Using the encoding above, define a function

$$\mathsf{unfold} : \forall\alpha(\alpha \to (\alpha \to (\mathsf{bool} \times \alpha)) \to \mathsf{stream})$$

[4 marks]

unfold can be defined as:

$$\Lambda\alpha.\lambda l : \alpha.\lambda n : \alpha \to (\text{bool} \times \alpha).\text{pack}(\alpha,(l,n))$$

(iv) Using unfold and the other functions above, define a function $\mathsf{notstream} : \mathsf{stream} \to \mathsf{stream}$, which returns a stream containing the boolean negation of the elements of the input stream. This answer should not use explicit pack or unpack expressions. [6 marks]

notstream can be defined as:

$$\lambda s : \text{stream.unfold stream } s(\lambda y : (\text{not}(\text{head } y), \text{tail } y))$$

[*Note*: You may use extensions to the pure polymorphic lambda calculus such as let-bindings, natural numbers, products, and sum types, but carefully note their use and their typing in your answers.]